Transitioning to Model Driven Software

Berner Architekten Treffen / CHOOSE
March 2007

OOPSLA'02

- *The era of expensive software development tools is long gone. The market already discards MDA tool-hype, and does not readily buy into MDA. However, domain-specific approaches and generator technology have a huge potential.*
- *The main problem is not one of tools - as some vendors would like people to think - but one of culture. Unless this is recognized, MDA will become the CASE of this decade. …*
- *The only realistic way to gain widespread acceptance for the use of MDA-type approaches and generation tools is by addressing the related cultural issues, and by providing tools at a very low cost - so low that a CEO or CTO does not have to think twice.*
- *How about an open source-based project? While the tool vendors jockey for position and battle amongst themselves, an open source project may actually deliver results and become a de-facto MDA standard.*

-- Jorn Bettin, Craig Cleaveland, Ghica van Emde Boas, Krzysztof Czarnecki

Hype

Object-orientation promised productivity and reuse

It didn’t happen

- Objects are useful
- But objects are too small-scale,
- they don‘t inherently support separation of technical and functional concerns,
- and humans are not good at using frameworks

Hype

Components again promised productivity and reuse

It again didn't happen

- Reuse of component containers did happen :-) example: EJB
- Component specifications as sets of operations are not enough
- Components tied to specific technologies

Hype

Model Driven Architecture promises productivity and reuse

It's not going to somehow magically happen this time

- The model-driven intention is encouraging
- Domain-specific notations evolve at a much slower rate than implementation technologies
- The "universal" aspect of UML is problematic
- The term "platform" is relative, yet many so-called MDA tools pre-define what is "the platform"
- The OMG QVT standard for model transformations is not yet widely adopted, and all commercial MDA tools use proprietary transformation technology

... unless we consider the other pieces of the software development puzzle

Model Driven Software

Bringing together several ideas

Concept

- Domain Engineering
- Design by Contract
- Generative Programming
- Iterative Development
- Commons

Buzz word compliant translation

- Software Product Lines
- Component Based Development
- Model Driven Development
- Agile Methodologies
- Open Source

Model Driven Software

A new set of values

- We prefer to validate software-under-construction over validating software requirements
- We strive to automate software construction from domain models; therefore we consciously distinguish between building software factories and building software applications
- We work with domain-specific assets, which can be anything from models, components, frameworks, generators, to languages and techniques
- We support the emergence of supply chains for software development, which implies domain-specific specialization and enables mass customization
- We see Open Source infrastructure software as leveling the playing field for economically viable software supply chains to emerge

All that was four to five years ago

So what has changed between then and now?

- There is still no market leading MDA tool vendor in sight
- Microsoft joined the club in 2003 with their Software Factories initiative
- Open Source MD* projects have emerged and have had time to mature
- For the most part decision-makers and software developers have been watching from the sidelines
  - The result of lessons learnt from Computer Aided Software Engineering (CASE) tools and from the productivity drop in the 90s that followed the adoption of object oriented languages such as C++ and Java.

Today

Misconceptions that are finally starting to go out of fashion

- *an agile approach is all you ever need*
- *objects & frameworks will solve your problems*
- *the world is a UML model*

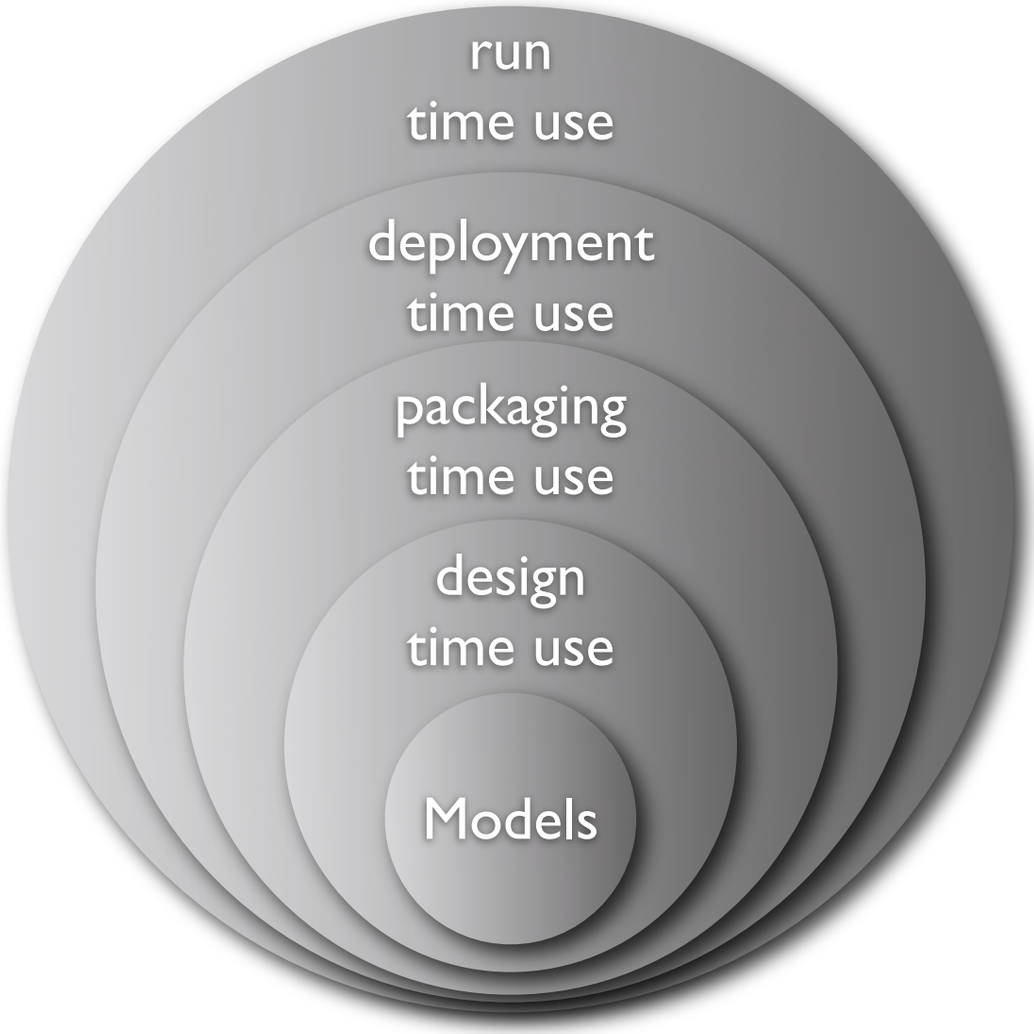
What is Model Driven Software?

The use of models can extend far beyond code generation

Models are used for

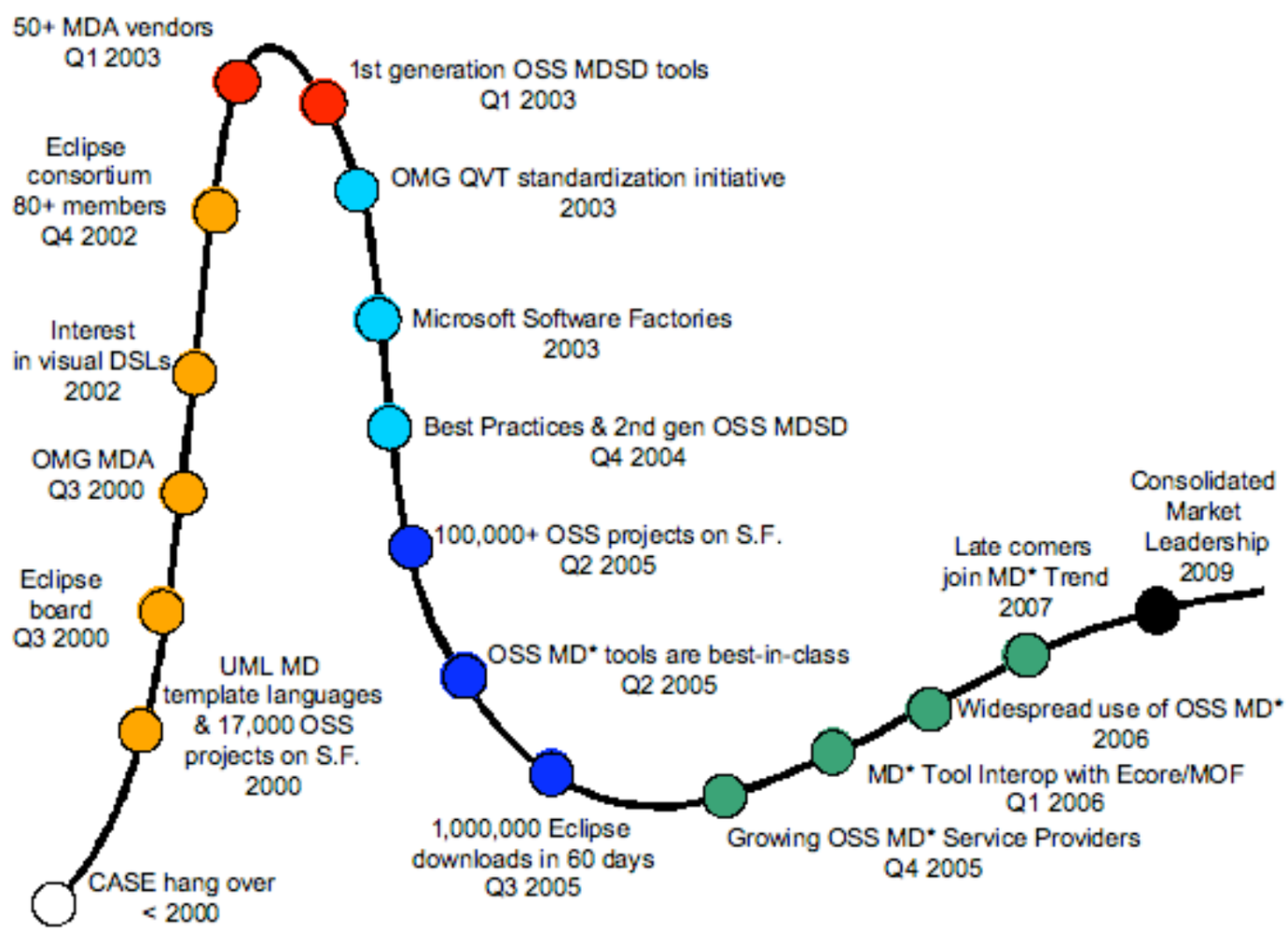
- Configuration via runtime interpretation of models & model driven applications
- Deployment of appropriate components into various environments and onto various hardware platforms
- Assembly and packaging of components
- Generation of code, tests, documents, ...

Only models that conform to a formal meta model are of interest



Putting some key events into perspective

The Model Driven & Open Source Software Hype Curve



The MD* trend is able to surf on the wave of mainstream acceptance of OSS tools that currently rolls through software development organizations

Domain Specific Modeling Languages

- The majority of software systems today are implemented in general-purpose languages that address a broad domain
- UML provide a formal visual syntax to describe object oriented software
- All advanced applications of model driven approaches involve domain specific languages (DSLs), i.e. incorporate domain concepts as first class language elements
- A DSL is close to the natural language of a domain expert, and specifications expressed in a DSL tend to be very compact
- Familiar DSLs are end user programming tools such as spreadsheets
- To solve the software productivity problem the OMG tries to raise the level of abstraction of its standards, via i.e. the adoption of MDA standards such as MOF and QVT. Although there is a grain of truth in this line of thought, it still suffers from the limitations of a one-fits-all approach

Standards

Yet another silver bullet

Interoperability?

- In theory interoperability should entail exchangeable implementations
- User pressure can easily lead to a premature agreement of a standard
- In the interest of generating sales, vendors will happily conform to a standard that does not cover important features
- The evolution of the XMI standard can be used to illustrate this point
  - The initial versions did not cover preservation of UML diagram layouts
  - This meant that XMI was only suitable for one-off data migration exercises
  - Even today interoperability between UML tools is not such that individual users in a team can work with different tools

Standardisation

A process riddled with difficulties

It starts with the recognising the need for a standard, then agreeing the scope of the standard, and finally, developing the standard

- The most successful standards apply to software that is widely considered a commodity. Standards such as HTML and TCP/IP provide the base for countless applications
- Bruce Perens, who announced "Open Source" to the world, and who published his first Open Source program in 1987, sums up the problem of consortia-based standard development as follows:
  - *In the consortium projects, there's always the handshake with one hand and a dagger in the other*
- In organisations that use rather than develop information technology solutions, there is a tendency to overestimate the benefits resulting from the adoption of standards

Standards easily take a decade to mature

Model transformation technology

What about emerging OMG MDA standards?

The problem

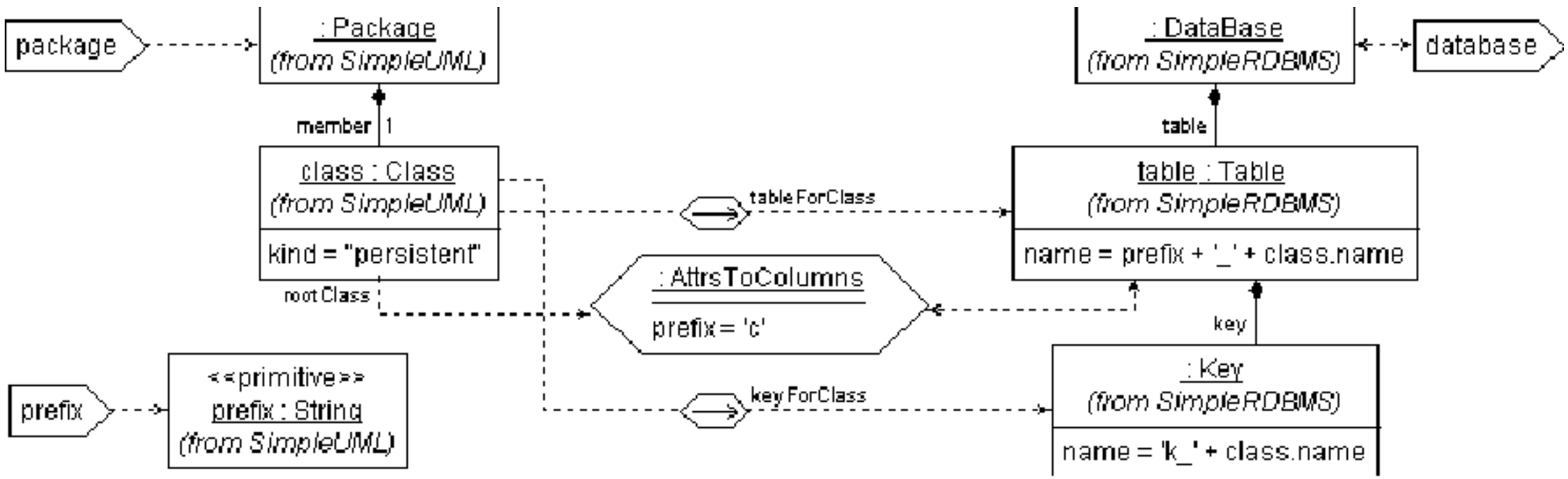
- If a textual template language is used to map directly from a highly abstract model to textual source code, then the structure of the model "drowns" in the textual template code that is structured along the lines of [textual] implementation code.

The "solution"

- Visual notations for transformations… they should have the advantage of being able to represent mappings between patterns in source and target models in the form of diagrams
  - This is the motivation behind visual languages such as UMLX and others
  - Hence also the new OMG Query View & Transformations (QVT) standard

Model transformation technology

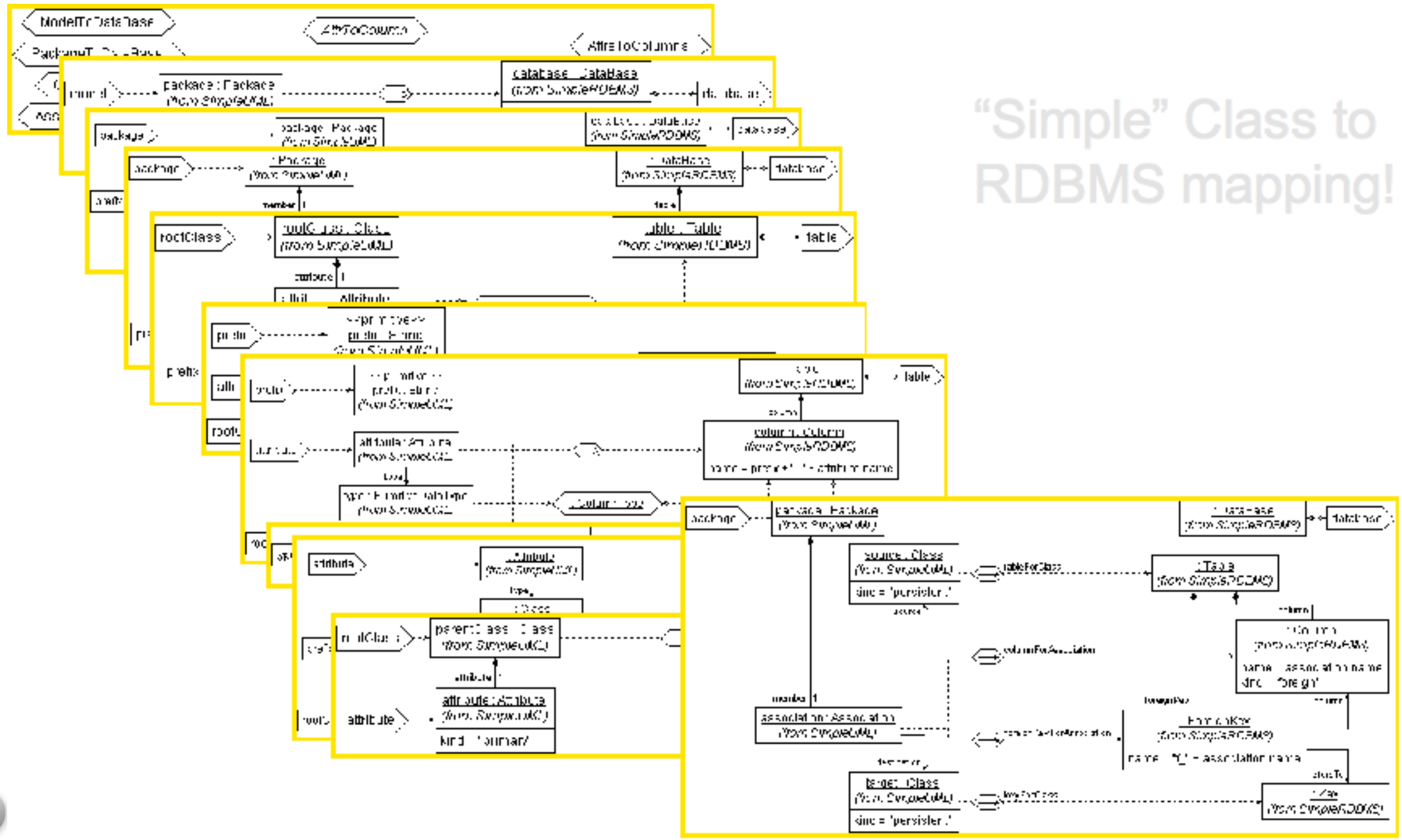
Problems with the "solution"



- Most concrete syntaxes (the notations that users would have to work with on a daily basis) proposed for visualizing model transformations are quite verbose
  - Any non-trivial transformation ends up being scattered across a multitude of diagrams
  - It becomes questionable whether the result is really better structured than a transformation expressed in a textual template language

Model transformation technology

How many transformation modelers does it take to change a light bulb?



Model transformation technology

The challenge

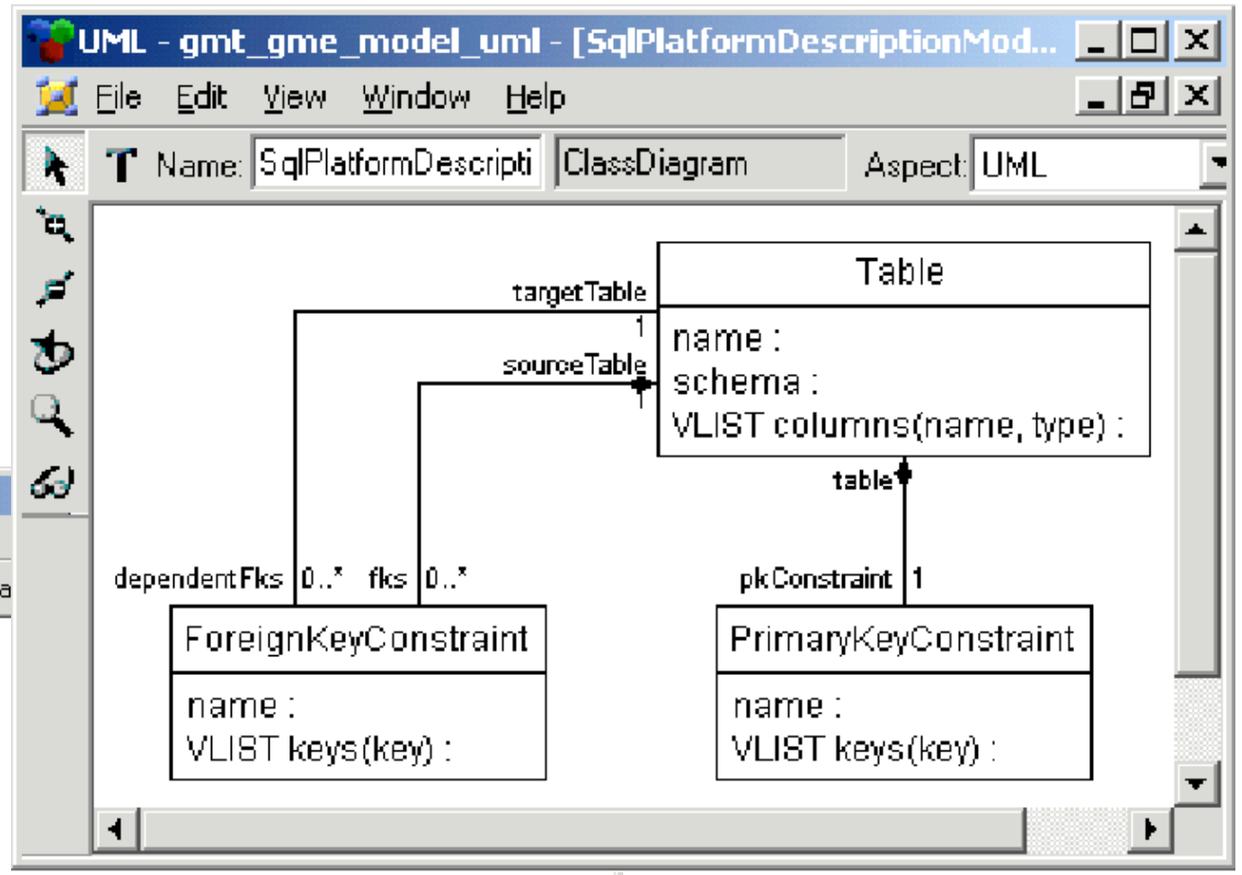
The answer lies in finding notations that allow the user to balance the ratio of visual / textual syntax

- UML class diagrams are the most popular type of diagram for visualizing software design
  - to a significant degree that is because users can choose how much detailed design information to include in textual form in the class boxes
- Visual representations are a very good tool to represent links between model elements
- Visual representations don't add value to the representation of linear sequences of model elements
- The optimal notation for a particular model transformation depends on the specific characteristics of the transformation, there is no good one-fits-all solution

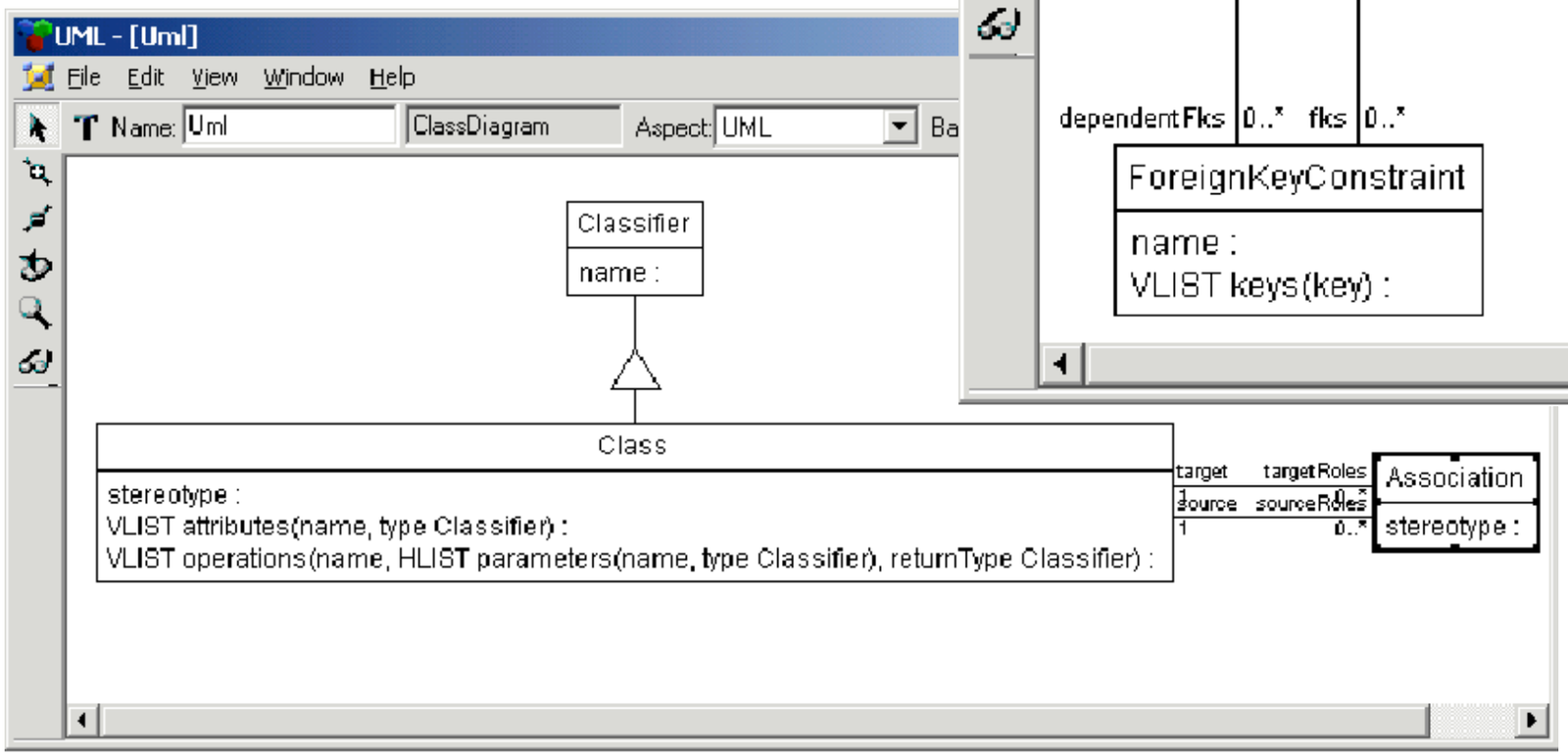
Model transformation technology

Example of possible alternative meta models for expressing O/R mappings

Platform Description Model

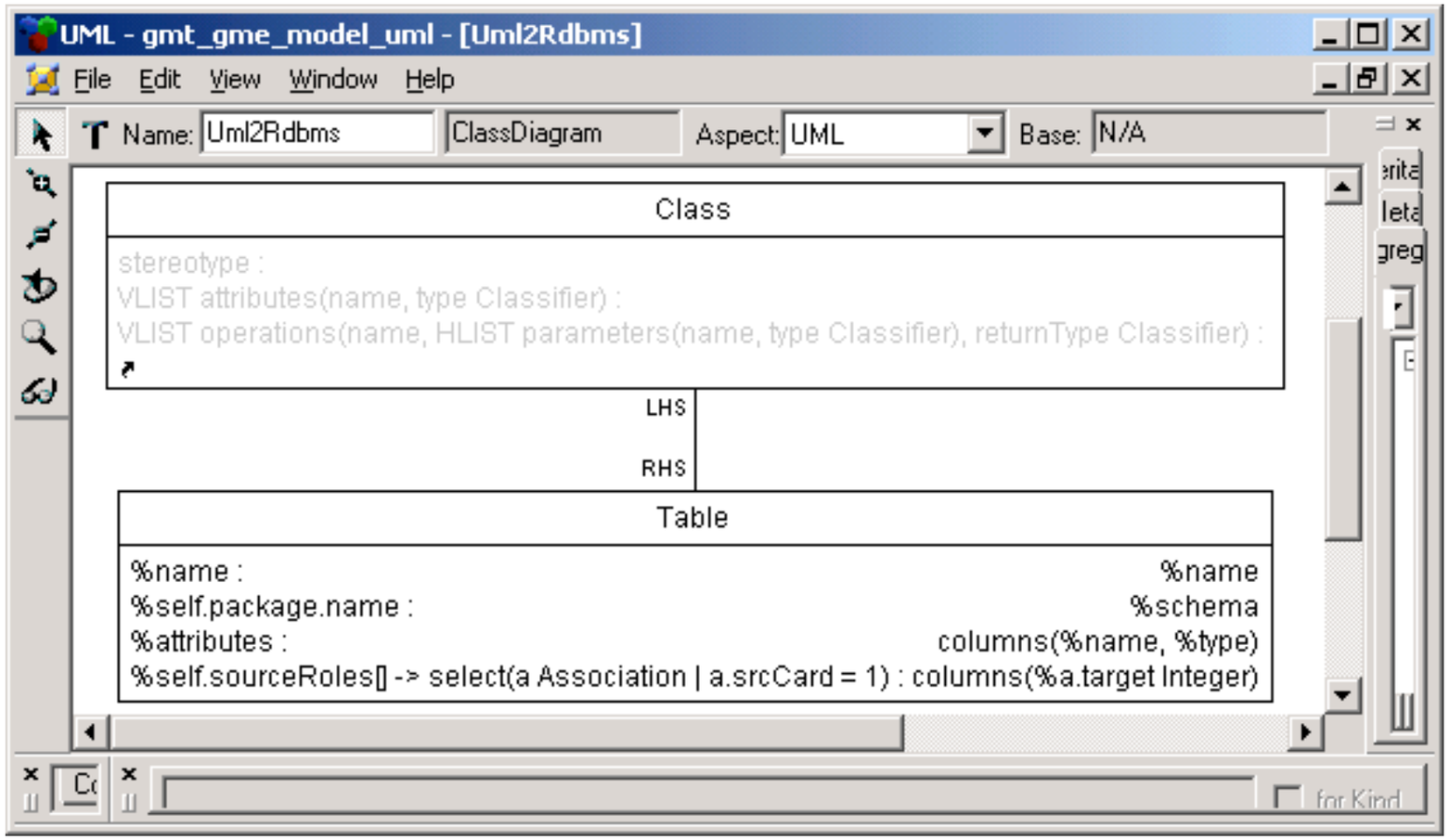


Platform Independent Model



Model transformation technology

PIM to PSM Transformation Model for O/R mappings



Why don't we see more use of DSLs?

The difficult aspect of development with DSLs is making appropriate DSLs available in the first place

- Sufficiently deep domain knowledge
- Practical experience in designing and implementing formal languages
- Good tools that enable fast implementation of DSLs, i.e. the creation of high quality editors and the creation of high quality compilers and/or interpreters
- Practical experience in integrating the use of one or more DSLs into an effective iterative and agile software development process that may also include handcrafted code written in general purpose languages
- Unjustified fears about not using a standards-based approach and mainstream programming languages

Meta Modeling

Keeping it simple

- When defining visual [domain specific] modeling languages, the key concepts that one always deals with are boxes, lines, roles, properties, and some structural container concept that enables grouping or classification of model elements
- Designing a DSL is nothing more and nothing less than identifying the names of the various types of boxes, lines, roles, properties, and containers that are intended to be allowable languages constructs, and subsequently precisely defining the semantics of these meta model elements
- The best way to validate whether such a simple meta meta model is sufficient is to apply it to express itself
  - It is also possible to express the MOF meta model or for that matter the UML meta model using the "boxes & lines" meta meta model
  - Tool vendor MetaCase was amongst the first to stress the need to keep meta modeling simple

Architecture-centric MDS Development

When there is no deep business domain knowledge available ...

- ... or where 90% of the domain amounts to "CRUD" functionality for maintaining business entities realized with an implementation technology stack of T1, T2, … Tn,
  - there is no point in trying to develop a DSL motivated by the business domain
  - That does not mean that in such a scenario there is no place for MDSD
  - It simply means that the main DSL could be a tailored type of "UML class diagram" that takes into account local constraints and limitations within the enterprise architecture of an organization
  - Further DSLs may come into play to describe workflow, visual characteristics and behavior of business entities on the user interface, mappings to structures from existing legacy systems, and so on

Architecture-centric MDS Development

Complexity & dependency management

- A quote from a senior software architect in a 1000+ person software organization (2003):

  Lack of dependency management is the biggest problem in our organization - this is not purely technical, it's a mindset thing

- A major weaknesses of many enterprise software systems is a lack of modularization
  - Architecture-centric MDSD is ideally suited to provide modularization constructs
  - that can already be applied during the elaboration of requirements
  - and that can automatically be enforced in the form of a strictly component based design within code
- The prospect of being able to <u>pro-actively manage and control dependencies</u> often is the key motivating factor to employ architecture-centric MDSD

Open source based MDS Development

Eclipse Modeling Framework & Ecore

Within the sphere of influence of the Eclipse platform Ecore is developing into a generally accepted common denominator for interoperability between MD* tools

- Ecore is a pragmatic interpretation of the MOF standard
- It is increasingly being used by various Eclipse based MDSD components:
  - The OpenArchitectureWare model driven template language
  - The Atlas Transformation Language
  - The MOFScript component for model to text transformations

There are many more OSS components for MDSD, and the list is growing

- Some OSS components are considered "best-in-class" by practitioners who have had the opportunity to apply both Open Source MD* tools and a range of proprietary MDA tools in industrial settings

Microsoft Software Factories

The Microsoft strategy does not fall in line with OMG standards and terminology

Software Factories amount to a DSL approach for Architecture-Centric MDSD

- Microsoft DSL tools are strongly coupled to the Microsoft Visual Studio IDE
- Software is increasingly developed by highly distributed teams that use the web as a collaboration platform
- A good tool chain needs to minimize all non-essential coupling
- The core integration points are source code artifacts (meta meta models, meta models, model transformations, models, and model elements) and open standards for exchanging these artifacts – and not tools
- It is unrealistic to expect that in the future all participants in a global software supply chain will be working with exactly the same proprietary tool set

The transition to Model Driven Software

Mapping out a path to safely reach the destination

Most software development organizations are still within the early phases of the journey towards MDS

- Building and using domain-specific tools and processes lead some people to believe that MDS is incompatible with Agile Software Development. This is one reason why some organizations continue to sit on the fence

Recommendation

- If you are still sitting on the fence, set up a small pilot project to explore MDS and judge for yourself. There is no substitute for first-hand experience

The transition to Model Driven Software

"Development" is just one aspect of Model Driven Software

The transition to MDS is akin to the paradigm shift from assembler coding to higher level languages

- The real issues relate to training and knowledge transfer in DSL design, and to the necessary changes in traditional software development processes

It is incorrect to assume that all DSLs are best packaged as part of an IDE ...

- ... and to assume that the target audience for DSLs is limited to software professionals
- Often building a good <u>model driven application</u> (i.e. an application that effectively is a powerful domain-specific modeling tool for end users) deserves as much or even more attention than the DSLs used in building the application

The transition to Model Driven Software

What transformation technology should you get started with?

Realistic expectations regarding MDA standards

- QVT is brand new, without an established user community
- Concerns about verbosity of visual notations for transformations still apply
- Today's proprietary and Open Source MDA/MDSD tools make use of non-industry-standard template languages
- However, extracting code templates is a small effort compared to hand coding a reference implementation (which needs to be done anyway) from which templates can be extracted

Recommendation

- It is not worthwhile losing sleep about a non-standardized template language. The ergonomics of the template language do count, but an accepted standard is still some time away. In any case, vendor lock-in can be avoided by using one of the Open Source options

The transition to Model Driven Software

The first MDSD project

Learn to walk before you run

- Like with any major paradigm shift, it is a good idea to first apply a new approach such as MDSD in a pilot project that involves a team that is motivated to make the transition
- The learning experience from a pilot project should be in place before rolling out MDSD to an entire organization

Recommendations

- Remember that a large part of MDSD is about knowledge management, and building domain-specific assets
- If you are new to meta modeling, start your MDSD pilot project with Architecture-Centric MDSD
- If your business is firmly rooted in the Microsoft technology stack, perhaps start with the Software Factories tools
- Further business domain-specific languages can be added later if required

The transition to Model Driven Software

Prerequisites

The sequence of steps does matter

- Designing useful DSLs depends on being able to pick up and articulate subtleties that may initially only be available in the form of tacit knowledge
- If fundamentals such as a disciplined agile requirements management process are not in place, then the subtleties that hold the key to making significant progress may be blocked out by the noise of daily emergencies

Recommendations

- Ensure that your team is capable of running a well oiled iterative and agile software development process before embarking on MDSD
- Otherwise your MDSD project may fail due to reasons that have nothing to do with MDSD in specific

The transition to Model Driven Software

Software organizations can be very conservative

Tangible results count and create confidence

- Introduction of a new process requires visible and unwavering management support

Recommendations

- If you have been around long enough, try to remember the organizational learning curve and the time required to move away from assembly language programming to modern higher level languages
- Don't assume that the transition to MDSD will be any easier, and prepare for the same types of detractors

The transition to Model Driven Software

Know your business

Domain knowledge

- No software development process can replace essential knowledge about a business
- The availability of domain experts with a deep understanding of your business is essential
- The potential of MDS is not delivered in a box that just needs to be unpacked

Recommendations

- Don't neglect the importance of a high quality reference implementation
- Domain-specific framework development can be a very substantial part of MDS
- Take an iterative, incremental approach at all times
- Keep the reference implementation up-to date at all times for the purpose of being able to test new increments of your MDSD tool kit

Managing software assets

MDS as a tool to improve your competitive edge

Domain-specific assets usually need to be built

- Only in some cases can they be bought off-the-shelf
- When only working towards quarterly results, then there is little incentive to build domain-specific assets, and quality of the original software design degrades significantly over time
- Unfortunately, treating software as a capital cost encourages this trend

Managing software assets

Assessing a software portfolio

A useful classification scheme for planning investments

- Strategic software assets—the heart of your business, assets that grow into an active human- and machine-usable knowledge base about your business and processes
- Non-strategic software assets—necessary infrastructure that is prone to technology churn and should be depreciated over two to three years
- Software liabilities—legacy that is a cost burden

Recommendations

- Strive to evolve your core business systems into strategic assets that increase in value over time through (re)use and refinement
- A healthy mix of strategic and non-strategic assets may sometimes only include a small core of strategic software assets
- Identify liabilities and either stop using them or replace them ASAP

Discussion & questions

Jorn Bettin
Managing Director
SoftMetaWare Ltd.

jorn.bettin@softmetaware.com
www.softmetaware.com

Tel +41 79 543 3767

Enabling Your Software To Evolve!

Montages Management GmbH
Stampfenbachstrasse 48
Postfach
CH-8035 Zürich

info@montages.com
www.montages.com

Tel +41 44 365 78 78
Fax +41 44 365 77 07

Montages™ faster development

Consulting for the financial industry